

Parallélisation de l'algorithme K-means

Arsen Pidburachynskyi

24 mai 2026

GitHub : [kmeans-openmp-cuda](#)

1 Introduction

Ce projet porte sur la parallélisation de l'algorithme K-means. K-means est un algorithme de clustering qui regroupe des points autour de plusieurs centroïdes. À chaque itération, chaque point est affecté au centroïde le plus proche, puis les centroïdes sont recalculés comme la moyenne des points qui leur sont associés.

L'algorithme est simple à formuler, mais il devient coûteux lorsque le nombre de points augmente. Pour n points, k clusters et une dimension d , l'étape d'affectation demande environ :

$$O(n \times k \times d)$$

opérations par itération. Cette étape est répétée plusieurs fois, ce qui rend K-means intéressant pour étudier la parallélisation.

Dans ce projet, quatre implémentations sont comparées :

- une version naïve en Python/NumPy ;
- une version parallèle sur CPU avec Cython et OpenMP ;
- l'implémentation `KMeans` de scikit-learn ;
- une version CUDA.

L'idée principale est de comparer une implémentation simple avec une version CPU parallélisée, puis de situer ces résultats par rapport à une bibliothèque existante et à une implémentation GPU.

2 Principe de l'algorithme

K-means alterne deux étapes :

1. affecter chaque point au centroïde le plus proche ;
2. recalculer chaque centroïde comme la moyenne des points affectés à son cluster.

Dans ce projet, les différentes versions utilisent la même initialisation : les k premiers points du dataset sont utilisés comme centroïdes initiaux. Ce choix permet de comparer les temps d'exécution dans un cadre contrôlé. Les différences de temps viennent donc surtout du calcul des distances et de la mise à jour des centroïdes, et non d'une initialisation différente.

Les quatre versions effectuent donc le même type de calcul. Les inerties finales sont globalement comparables, surtout entre OpenMP, scikit-learn et CUDA. Des écarts peuvent apparaître à cause de la précision numérique, de l'ordre des additions dans les réductions parallèles, ou du fait que certaines versions s'arrêtent après un nombre légèrement différent d'itérations.

3 Implémentations

3.1 Version naïve NumPy

La première version est écrite en Python avec NumPy. Elle suit directement les étapes de l'algorithme : calcul des distances, affectation des points, puis mise à jour des centroïdes.

Cette version sert de base de comparaison. Elle n'est pas explicitement parallélisée dans le code.

3.2 Version CPU parallèle

La version parallèle est écrite en Cython et utilise OpenMP. Deux parties sont parallélisées.

La première est l'affectation des points aux clusters. Chaque point peut être traité indépendamment des autres. La boucle sur les points est donc parallélisée avec `prange`.

La deuxième partie est la mise à jour des centroïdes. Cette étape est moins directe à paralléliser, car plusieurs threads peuvent vouloir ajouter des valeurs au même cluster. Pour éviter les conflits d'écriture, chaque thread utilise des accumulateurs locaux. Ensuite, une réduction finale additionne les résultats locaux.

Le principe est le suivant :

```
pour chaque thread:
    accumuler les sommes locales par cluster
    compter les points locaux par cluster
```

```
après la boucle parallèle:
    additionner les sommes locales
    calculer les nouveaux centroïdes
```

Cela évite que plusieurs threads écrivent directement dans la même mémoire au même moment.

3.3 Scikit-learn

Scikit-learn est utilisé comme implémentation existante de référence. Il s'agit du même principe général de K-means, mais dans une bibliothèque plus complète et plus robuste.

La comparaison avec scikit-learn permet de situer les performances de notre version OpenMP. Elle ne doit pas être interprétée comme une affirmation générale que notre code est meilleur que scikit-learn. Notre implémentation est plus spécialisée : elle traite des matrices denses numériques, avec une initialisation fixée et peu d'options.

3.4 Version CUDA

La version CUDA suit le même algorithme K-means, mais l'exécution est organisée différemment. Sur CPU, la parallélisation se fait avec quelques threads OpenMP. Sur GPU, on utilise beaucoup plus de threads CUDA, regroupés en blocs.

Pour l'étape d'affectation, chaque thread CUDA traite un ou plusieurs points. Il calcule les distances aux centroïdes, garde le plus proche, puis écrit le label du point. Cette partie est très adaptée au GPU, car les points sont indépendants.

La mise à jour des centroïdes utilise une idée proche de la version OpenMP, mais à un autre niveau. Dans la version OpenMP, chaque thread CPU possède ses accumulateurs locaux. Dans la version CUDA, ce sont les blocs CUDA qui accumulent d'abord des sommes locales en mémoire partagée. Ensuite, ces sommes locales sont ajoutées aux sommes globales, puis les nouveaux centroïdes sont calculés.

La différence principale est donc :

- OpenMP : réduction locale par thread CPU ;
- CUDA : réduction locale par bloc CUDA, en mémoire partagée.

La version CUDA permet donc de comparer la version CPU parallèle avec une version utilisant un parallélisme beaucoup plus massif.

4 Protocole expérimental

Les tests sont effectués sur des datasets synthétiques numériques. Deux tailles sont utilisées :

1 000 000 et 10 000 000 points

La dimension est fixée à 16. Les nombres de clusters testés sont :

$$k \in \{2, 4, 8, 10\}$$

Le nombre maximal d'itérations est fixé à 30. Les implémentations peuvent s'arrêter avant si les affectations ne changent plus ou si les centroïdes se stabilisent.

Pour chaque configuration, on mesure :

- le temps total ;
- le nombre réel d'itérations ;
- le temps moyen par itération ;
- l'inertie finale.

Le temps total donne le coût réel d'une exécution complète. Le temps moyen par itération est aussi utile, car toutes les implémentations ne font pas toujours exactement le même nombre d'itérations.

5 Résultats

5.1 Vue globale des temps

La figure 1 montre le temps total d'exécution pour les quatre implémentations. L'échelle logarithmique est utilisée car les temps CUDA sont beaucoup plus faibles que les temps CPU.

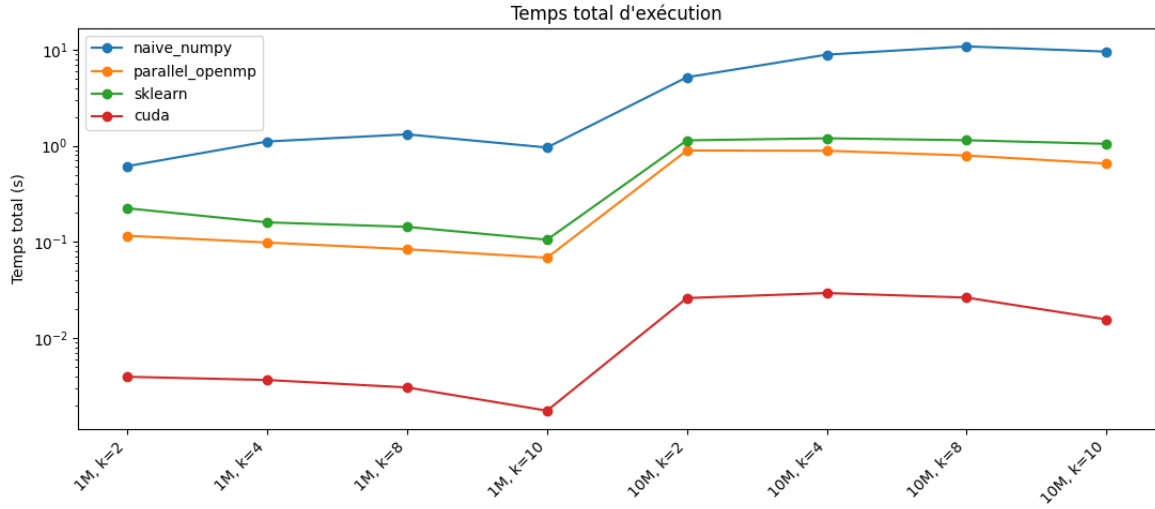


FIGURE 1 – Temps total d'exécution pour les quatre implémentations

Remarque : pour chaque configuration, OpenMP, scikit-learn et CUDA effectuent le même nombre d'itérations. La version naïve s'arrête parfois une itération plus tôt, mais reste malgré cela plus lente.

On voit directement que la version OpenMP accélère fortement la version naïve. Scikit-learn donne aussi des temps rapides, mais la version OpenMP est compétitive sur ce benchmark. CUDA est beaucoup plus rapide, ce qui est attendu pour ce type de calcul très parallèle.

5.2 Tableau des temps

Le tableau 1 donne les temps d'exécution principaux. Il résume les résultats visibles sur la figure précédente.

Points	Clusters	Naïf (s)	OpenMP (s)	sklearn (s)	CUDA (s)
1 000 000	2	0.613	0.116	0.223	0.00395
1 000 000	4	1.107	0.099	0.160	0.00366
1 000 000	8	1.313	0.084	0.143	0.00306
1 000 000	10	0.958	0.068	0.105	0.00176
10 000 000	2	5.154	0.893	1.138	0.0261
10 000 000	4	8.860	0.887	1.195	0.0294
10 000 000	8	10.792	0.792	1.141	0.0263
10 000 000	10	9.521	0.653	1.046	0.0156

TABLE 1 – Temps total d'exécution des quatre implémentations.

La version OpenMP est plus rapide que la version naïve dans toutes les configurations. Elle est aussi plus rapide que scikit-learn sur ce benchmark. Cela ne veut pas dire que l'implémentation est meilleure que scikit-learn de manière générale, mais cela montre que la version OpenMP est efficace dans ce cadre précis.

5.3 Speedups

Le tableau 2 donne les speedups les plus importants : OpenMP par rapport à la version naïve, scikit-learn par rapport à OpenMP, et CUDA par rapport à OpenMP.

Dans la colonne sklearn/OpenMP, une valeur supérieure à 1 signifie que scikit-learn est plus lent que la version OpenMP sur cette configuration.

Points	Clusters	OpenMP vs naïf	OpenMP vs sklearn	CUDA vs OpenMP
1 000 000	2	5.29	1.93	29.29
1 000 000	4	11.24	1.62	26.89
1 000 000	8	15.64	1.71	27.40
1 000 000	10	14.01	1.54	38.94
10 000 000	2	5.77	1.27	34.27
10 000 000	4	9.99	1.35	30.18
10 000 000	8	13.63	1.44	30.07
10 000 000	10	14.59	1.60	41.77

TABLE 2 – Speedups obtenus sur les différentes configurations.

Le speedup OpenMP est toujours supérieur à 5, et atteint environ 15 dans certains cas. Le gain augmente généralement avec le nombre de clusters. C'est cohérent avec le coût de K-means : plus il y a de clusters, plus il faut calculer de distances pour chaque point.

La comparaison avec scikit-learn montre que la version OpenMP est compétitive. Sur ces tests, scikit-learn est entre 1.27 et 1.93 fois plus lent que la version OpenMP. Cette différence vient probablement du fait que notre code est plus spécialisé : il ne gère qu'un cas simple, dense, avec une initialisation fixée.

CUDA est entre environ 27 et 42 fois plus rapide que la version OpenMP. Ce résultat est cohérent avec la nature du calcul : beaucoup de distances indépendantes sont calculées, ce qui convient très bien au GPU.

5.4 Qualité des résultats

Comme les implémentations utilisent le même algorithme et la même initialisation, les résultats de clustering doivent être proches. Pour le vérifier, on compare l'inertie finale, c'est-à-dire la somme des distances quadratiques entre les points et leurs centroïdes.

Le tableau 3 donne quelques exemples d'inerties obtenues. Les valeurs sont très proches entre OpenMP, scikit-learn et CUDA. Cela confirme que les différentes implémentations produisent un clustering comparable.

Points	Clusters	OpenMP	sklearn	CUDA
1 000 000	2	5.1341×10^8	5.1330×10^8	5.1341×10^8
1 000 000	10	1.3202×10^6	1.3206×10^6	1.3200×10^6
10 000 000	2	5.1334×10^9	5.1254×10^9	5.1341×10^9
10 000 000	10	1.3306×10^7	1.3303×10^7	1.3200×10^7

TABLE 3 – Exemples d'inerties finales pour OpenMP, scikit-learn et CUDA.

Les écarts restants viennent surtout de détails numériques : précision flottante, ordre des additions, et réductions parallèles. La version naïve n'est pas affichée dans ce tableau, car elle s'arrête parfois une itération plus tôt sur les grands jeux de données, ce qui rend son inertie légèrement moins comparable dans ces cas.

Le but du benchmark est donc bien de comparer les performances d'exécution entre des implémentations du même algorithme, et non de comparer plusieurs méthodes de clustering différentes.

6 Analyse

Les résultats montrent que la parallélisation CPU est efficace pour K-means. L'étape d'affectation est naturellement parallèle, car le calcul du cluster d'un point ne dépend pas des autres points.

La mise à jour des centroïdes demande plus d'attention. Une version naïve parallèle pourrait provoquer des conflits, car plusieurs threads peuvent écrire dans le même cluster. L'utilisation d'accumulateurs locaux par thread règle ce problème et rend cette étape parallélisable.

Le speedup OpenMP augmente avec le nombre de clusters. Cela s'explique par le fait que chaque point doit être comparé à davantage de centroïdes. La partie parallélisée devient donc plus importante dans le temps total.

La comparaison avec scikit-learn est aussi intéressante. Sur ce benchmark, la version OpenMP est plus rapide. Je ne l'interprète pas comme une victoire générale contre scikit-learn, car scikit-learn est une bibliothèque complète, avec beaucoup plus de fonctionnalités et de vérifications. Ici, le résultat montre plutôt qu'une implémentation spécialisée et parallélisée peut être très efficace sur un cas contrôlé.

CUDA donne les meilleurs temps. Ce n'est pas surprenant : K-means contient un grand nombre de calculs indépendants, ce qui correspond bien au modèle GPU. Cette comparaison permet de voir que le même problème peut profiter de plusieurs niveaux de parallélisme.